

Design Decisions

This file collects the non-obvious choices made while building this system, with the reasoning behind each one. It exists because the person building this needs to be able to explain every part of the codebase, not just that it runs so anywhere a reasonable person could disagree with a choice made here, that choice is written down.

Important note on how this was built and verified

This codebase was originally written without a local Python interpreter or Docker available in the build environment. Every design and API claim about **gym-pybullet-drones** below (class names, constructor signatures, the control-loop pattern, the exact license) was verified by fetching the actual source of `gym_pybullet_drones/examples/pid.py`, `pyproject.toml`, and `LICENSE` from the upstream repository at <https://github.com/utiasDSL/gym-pybullet-drones> (pinned to commit `9bc12bc583fa3b28807b2f90a8cadf09fb06e1ff`) rather than from memory. The pure Python logic (schema, validator, formation) was checked by hand-tracing the arithmetic for representative inputs (see the numeric examples in this file) before it was ever run.

It has since been executed and verified on Linux: `pytest tests/` passes, `docker build + the headless docker run demo work`, and the full `--prompt → Ollama → validate_mission → formation → --gui` PyBullet path has been run repeatedly for both example prompts, with two of those runs recorded as the demo videos in `demo/` (see `docs/WRITEUP.md` for the video-to-prompt mapping and the observed local-inference timing).

1. Schema: anyOf Discriminated Union Instead of if/then

`mission_schema.json`'s `task` field originally used `if/then/allOf` to make `area_of_interest` required only when `task.type == "area_sweep"` (and `route` only for `"patrol_route"`). That was changed to an `anyOf` of two fully-specified variant objects (each `additionalProperties: false`, each with its own `required` list gated by a `const` on `type`).

Why: Claude's structured-output enforcement (`output_config.format`) supports `anyOf/const/$ref` but does not document support for `if/then`. Using `anyOf` means the JSON Schema handed to `output_config.format` in `llm_planner/planner.py` and the JSON Schema handed to `jsonschema` in `schema/validator.py` can be the exact same file, with no need to strip conditionals for one consumer and not the other.

2. regroup_point: Required-but-Nullable, Not Optional

Every `task` variant lists `regroup_point` in `required`, with a value type of `anyOf: [point2d, null]`, rather than making it an optional property.

Why: structured-output strict-schema enforcement does not reliably support optional (non-required) object properties most strict-schema systems, this one included, expect every property to be listed

in `required`, using a nullable type for “value not applicable” instead of omission. Making it required-but-nullable keeps the schema strict-mode compatible without weakening what it actually enforces: `schema/validator.py`’s `validate_mission()` still rejects a “split” formation whose `task.regroup_point` is null.

3. “split” Is a Routing Strategy, Not a Geometric Shape

The take-home’s spec groups `wedge/line/column/split` together under one `formation.type` enum. Implementation-wise they aren’t the same kind of thing: `wedge/line/column` hold a constant geometric offset from a shared reference path (`formation/geometry.py`); `split` divides the task into distinct per-drone segments that reconverge (`formation/route_split.py`). The enum stays literally as specified, but `formation_planner.plan_formation()` branches on it early and routes to one of two completely different computations there’s no shared “formation geometry” code path for `split`.

4. Two-Stage Validation

`schema/validator.py` has two families of check, and they run at different points in the pipeline:

- **Stage 1** (`validate_mission`) runs on the LLM’s raw output, before anything else touches it: JSON Schema structure, then cross-field safety checks (altitude/speed vs. safety ceilings, `spacing_m` vs. `min_separation_m`, geofence containment of the *raw task*’s points, degenerate-polygon/degenerate-route rejection).
- **Stage 2** (`validate_formation_separation`, `validate_formation_geofence`) runs after the formation layer has computed concrete per-drone paths, and is called from `executor/executor.py` never from stage 1.

Why it can’t be one stage: “do any two drones’ paths get too close” and “does every drone’s *actual* flown path stay in the geofence” are both questions about per-drone trajectories, and those don’t exist until after `formation.plan_formation()` runs. The raw mission JSON only describes one shared task (a route or an area) and a formation type there’s nothing to check separation or per-drone geofence containment *against* until the formation layer turns that into N distinct paths.

4a. Why Stage 1’s Geofence Check Isn’t Enough on Its Own

Stage 1’s geofence check only ever sees the task’s own points (the route, the area of interest, the regroup point). It has no visibility into what the formation layer will do with them. A wide `wedge` with several ranks and a large `spacing_m` can offset its outer wingmen well outside a geofence that the shared reference path never leaves stage 1 would pass such a mission, and only `validate_formation_geofence` (stage 2) catches it. This was found by hand-tracing the wedge offset math for the `demo/example_missions/wedge_sweep.json` example while writing it, not by running code (see the note at the top of this file) worth calling out because it’s exactly the kind of gap that’s easy to miss when a schema-shaped raw JSON check *feels* like it should be sufficient.

4b. The Final-Waypoint Exemption in `validate_formation_separation`

Split formations converge to the *exact same point* (`task.regroup_point`) at the end of every drone’s path, by construction that’s what “regroup at the end” means. A naive pairwise-distance check across every waypoint index would flag that convergence as a separation violation on ev-

every single split mission, which would make the feature the operator explicitly asked for unusable. `validate_formation_separation` therefore excludes the final waypoint index from its checks. This doesn't weaken the check for `wedge/line/column`: those formations hold a constant, schema-guaranteed non-zero offset (`spacing_m` has `exclusiveMinimum: 0`) at *every* waypoint including the last, so there's nothing that only fails at that one index for them. The function stays agnostic to *why* the last waypoint might coincide across drones it just always exempts it, which is the simplest rule that covers the real case.

5. Structured Outputs Is a Convenience, Not the Safety Mechanism

`llm_planner/planner.py` calls Claude with `output_config.format` set to `mission_schema.json`, which constrains the model's output shape at generation time. This is *not* a substitute for `schema/validator.py`: structured-output enforcement does not reliably enforce numeric bounds (`minimum/maximum/exclusiveMinimum`), string-length bounds (`minLength`), or array-length bounds (`minItems`) per Claude's own documented JSON Schema limitations, those are stripped/best-effort on the API side. A response can be perfectly schema-shaped and still violate `altitude_m <= safety.max_altitude_m`, or have a 2-point "polygon." Every mission structured-output-constrained or not goes through the full `validate_mission()` gate before anything is built or simulated. If Claude's structured-output feature were removed entirely, this system would still be correct, just slower to reject a malformed response (it'd get one round trip further before `validate_mission` caught it instead of the API layer).

6. Coordinates Are Local XY Meters, Not Lat/Lon

The schema uses plain `{x, y}` meters in an unspecified local frame, not geodetic coordinates. This matches gym-pybullet-drones' own coordinate convention (a local ENU-style simulation frame) and keeps the LLM's job simpler generating "a sensible 120x60 meter rectangle" is a much easier target than generating georeferenced coordinates for a sim that has no concept of geography. If this system ever needed to map to real-world GPS-guided drones, a lat/lon $\leftrightarrow$ local-frame transform would sit at the `llm_planner/executor` boundary, not inside the schema itself.

7. `sweep_path_for_area`: Single-Pass Sweep, Not Full Lawnmower Coverage

`formation/geometry.sweep_path_for_area()` reduces an `area_sweep` task to a single straight line through the polygon's bounding-box centerline, along its longer axis. A `wedge/line/column` formation flown along that line still covers a swath roughly `num_agents * spacing_m` wide in one pass a reasonable read of "sweep this area in a wedge" for a system this size but it is **not** true multi-lane boustrophedon coverage clipped to the polygon boundary. Building a real lawnmower-pattern coverage planner (parallel lanes, polygon clipping, turn-around geometry at each lane end) is a meaningfully bigger scope than this take-home's stated goals, and doing it half-right would be worse than a documented, isolated simplification. If this needs to be extended, it's a single-function change: `sweep_path_for_area()`'s signature and its caller in `formation_planner.py` don't need to change, only its internals.

8. Formation Offsets Are Computed in the Path’s Local Tangent Frame

`formation/geometry._offset_path()` doesn’t apply a fixed world-frame (dx , dy) offset to every point it computes the reference path’s local direction of travel at each point and applies lateral/-longitudinal offsets *relative to that direction*, then rotates into world XY. This means a wedge or line formation holds its shape through a turn in the reference path, not just on a straight line. The rank-offset formulas (`_rank_offsets`) are:

Formation	Rank r (0 = leader) offset
line	side-by-side, alternating left/right: $\text{lateral} = \pm \text{ceil}(r/2) * \text{spacing_m}$, $\text{longitudinal} = 0$
column	nose-to-tail: $\text{lateral} = 0$, $\text{longitudinal} = -r * \text{spacing_m}$
wedge	both at once: $\text{lateral} = \pm \text{ceil}(r/2) * \text{spacing_m}$, $\text{longitudinal} = -\text{ceil}(r/2) * \text{spacing_m}$

These were hand-verified for `spacing_m = 5` and a horizontal reference path in the `docstrings/tests` of `formation/geometry.py` and `tests/test_formation_geometry.py` (right = tangent rotated -90 , so heading $+x$ gives right = $-y$).

9. route_split: Distance-Based Splitting, Fixed Resample Density

`formation/route_split.split_path()` divides a route by cumulative *distance*, not waypoint count a 3-waypoint L-shaped route isn’t split “first point / rest,” it’s split into equal-length flying segments, preserving any original waypoints that fall inside a segment (so a bend inside one drone’s segment is still tracked, not flattened into a straight cut). Every segment is then resampled to a fixed `RESAMPLE_POINTS_PER_SEGMENT` (6) points.

Why a fixed resample count: `schema.validate_formation_separation` and the executor’s lockstep sim loop both assume every drone’s waypoint list is the same length. Distance-based splitting alone doesn’t guarantee that (two segments of equal length can still need different numbers of *original* waypoints to represent); resampling to a fixed count does, cheaply, at the cost of not preserving every original vertex exactly (interior vertices are kept as resample anchors where they fall, but the resample can still smooth sharp features between anchors if `RESAMPLE_POINTS_PER_SEGMENT` is too low for a very kinked segment reasonable for the route complexity this system expects from an operator’s natural-language description).

10. Determinism, End to End

“Same JSON in, same behavior out” is enforced by construction, not by convention:

- `formation/` and `executor/` contain no calls to `random`, no wall-clock-dependent branching, and no reliance on dict/set iteration order for anything numeric (Python dicts are insertion-ordered, and every dict here is built from `agent_ids_for()`, which is itself deterministic).
- Initial drone positions (`initial_xyzs` in `executor.build_target_trajectories`) are derived from the formation plan’s own first waypoints, never from an arbitrary/random starting arrangement.

- `sim/sim_runner.py` uses `Physics.PYB` (pure PyBullet rigid-body dynamics) rather than a physics mode with aerodynamic ground-effect/drag/downwash perturbations, and PyBullet’s DIRECT-mode physics is itself deterministic given identical inputs.
- The one deliberately non-deterministic thing in the whole pipeline is the LLM call itself (`llm_planner/planner.py`) which is exactly the point: the LLM’s non-determinism is fenced off to a single “propose a plan” step that produces a JSON artifact, and every step after that JSON exists is a pure function of it.

11. Executor Synchronizes Per-Drone Travel Duration to the Slowest Agent

`executor.build_target_trajectories()` computes each agent’s own path length / `speed_mps` as its “natural” lap duration, then resamples **every** agent’s path to the number of control steps implied by the *longest* of those durations. A drone with a shorter path (e.g., the inside of a wedge turn, or a shorter `split` segment) ends up moving slower than the mission’s nominal `speed_mps` to stay time-aligned with the rest of the squad.

Why: this is what real formation flying looks like the drone with the shortest path to fly slows down to hold position relative to the others, rather than each drone independently honoring `speed_mps` and arriving at different times (which would silently break the “hold formation” premise the operator asked for). It also keeps `validate_formation_separation`’s lockstep-index assumption trivially true without extra bookkeeping.

12. jsonschema Only, Not jsonschema + a Parallel pydantic Model

The stack allows “pydantic (or jsonschema).” This project uses `jsonschema` validation against `mission_schema.json` as the single source of truth, with mission data flowing through the rest of the pipeline as plain dicts, rather than also defining `pydantic` models that mirror the schema.

Why: maintaining two representations of the same contract (a JSON Schema file and a set of Pydantic classes) means every schema change has to be made twice and kept in sync by hand a real risk given `mission_schema.json`’s `anyOf` discriminated union, which is exactly the kind of construct that’s easy to get subtly wrong translating between the two representations, especially without being able to run a test suite to catch drift during development. One canonical schema file, validated by one library, was the lower-risk choice here.

13. gym-pybullet-drones Integration: What’s Cited vs. Original

`sim/sim_runner.py`’s docstring credits `gym_pybullet_drones/examples/pid.py` (MIT License, Copyright © 2020 Jacopo Panerati, <https://github.com/utiasDSL/gym-pybullet-drones>) for: the `CtrlAviary + DSLPIDControl + Logger` combination and their import paths, the “`env.step()` first using the previous action, then compute the next action from the returned obs” loop ordering, and the `target_pos/target_rpy` keyword shape passed into `DSLPIDControl.computeControlFromState`. These were confirmed by fetching the actual file from GitHub (see the note at the top of this file), not recalled from training.

Everything else in `sim/` and `executor/` is original to this project: building time-indexed target trajectories from mission JSON via the formation layer’s own `resample_polyline`, sourcing initial

positions from the mission plan instead of an arbitrary circular arrangement, tiling a single lap for `loop_count` repeats, and the minimum-observed-separation report.

`requirements.txt` pins `gym-pybullet-drones` to a specific commit (9bc12bc583fa3b28807b2f90a8cadf09fb06e1f on `main`) rather than the package’s most recent tag (`v1.0.0`) or a bare `main` reference. The `v1.0.0` tag predates the Gymnasium-based rewrite this project depends on (5-tuple `env.step()` returns; `CtrlAviary/DSLPIDControl/Logger` in their current form) pinning to it would install an incompatible, older API. Tracking `main` unpinned would make builds non-reproducible as upstream changes. A specific commit is the only option that’s both correct and stable.

14. LLM Backend: Ollama Instead of the Anthropic API

The task spec calls for the Anthropic API as the LLM layer, and that’s what `llm_planner/planner.py` originally called. It was switched to a locally-hosted model served by [Ollama](#) (default `qwen2.5:7b`) because no Anthropic API credit was available to actually run this system end to end. This is a documented substitution, not a silent scope change the original architecture’s whole premise (the LLM boundary is isolated to one file that does “prompt in, JSON out” and nothing else) is exactly what makes this swap cheap and low-risk to make.

What changed: only `llm_planner/planner.py`. The Anthropic SDK client call was replaced with a plain HTTP POST `/api/chat` to a local Ollama server (stdlib `urllib.request`, no new dependency), and Ollama’s own `format` request field is used to pass `mission_schema.json` for schema-constrained decoding the same intent as the Anthropic version’s `output_config.format`, kept for consistency across backends. A JSON-parse retry loop (`MAX_JSON_RETRIES = 2`) was added because local 7-8B models are less reliable than Claude’s structured-output enforcement at actually emitting parseable JSON, even with `format` set; the retry is scoped specifically to “the response didn’t parse as JSON,” re-prompting the model with its own bad output plus a corrective instruction. It does not retry connection failures or HTTP errors from Ollama itself (server not running, model not pulled) those aren’t transient JSON-quality problems, and retrying them just delays a clear error message.

What did NOT change, and why that’s the point: `mission_schema.json`, `schema/validator.py`, `formation/`, `executor/`, and `sim/` are all untouched. None of them import anything LLM-related or know which backend produced the mission JSON they’re handed the two-stage validation gate (§4) treats a schema-valid, safety-compliant JSON object identically regardless of whether Claude or a local Qwen model wrote it. `run.py` also didn’t change: `plan_mission(prompt, **kwargs)` kept the same call shape (the `model` kwarg now means an Ollama model tag instead of a Claude model ID, but the call site doesn’t need to know that).

Reverting to the Anthropic API, if credit becomes available, is a single-file change: restore `llm_planner/planner.py`’s previous client call (construct `anthropic.Anthropic()`, call `client.messages.create` with `output_config.format` set to the schema, same `SYSTEM_PROMPT` none of that needs to change either). Nothing downstream of `plan_mission()` needs to be touched, tested differently, or even be aware a backend swap happened. This is the payoff of having drawn the LLM/deterministic-pipeline boundary as strictly as `docs/architecture.md` describes: swapping *which* model fills in the JSON template is a change contained entirely on one side of that boundary.

Model choice tradeoff: `qwen2.5:7b` over a 70B-class local model. This system’s LLM-facing task is narrow and close to template-filling (map a short natural-language sentence onto a fixed JSON schema with a handful of enums and coordinate lists), not open-ended reasoning a 70B model’s extra

reasoning capacity buys little here, while costing 40GB+ of RAM/VRAM most desktops don't have and much slower inference. `qwen2.5:7b` (~4.7GB, already instruction-tuned under Ollama's default tag) was chosen over `llama3.1:8b` (~4.9GB, also instruct-tuned by default) specifically because Qwen2.5's training emphasizes structured/tool-call-style output more than Llama 3.1's does, which is the one property that matters most for this task's reliability. Both tags were verified against Ollama's own model library pages rather than guessed, since a wrong tag would just break the first `ollama pull`.